

Building Resilient Distributed Systems - StudySync

M Zohaib Sajjad - bscs23098 | PDC Assignment 4

Part 1 - Analysis: where the naive architecture breaks

Problem 1 - Lost update on concurrent document edits

The sync issue lives in the read-modify-write window inside the API request lifecycle. Alice reads document *d1* at version 1 into her client; Bob does the same a moment later. Both clients now hold the same snapshot. Alice PUTs her edit, the backend issues an unconditional UPDATE documents SET content = ?, and writes through. Bob then PUTs his edit and his unconditional UPDATE overwrites Alice's row. The database commits the second write happily because nothing in the statement asserts what Bob believed the prior state to be. No row-level lock, no version check, no serializable isolation - the second writer wins and the first writer's content vanishes silently. The classical Lost Update anomaly.

Problem 2 - Dropped Clerk webhook leaves users in a divergent state

Clerk delivers a subscription cancellation as a fire-and-forget HTTPS POST. The naive handler treats each POST as a one-shot command: parse, mutate the user row, return 200. There is no idempotency table keyed on the *svix_id*, no outbox/inbox pattern, and no retry queue. If a network blip drops the TCP segment carrying the webhook (or our app crashes mid-request), Clerk's delivery is lost from our side and our row never transitions from *premium* to *free*. Worse, even when Clerk retries it (and it does), the naive handler would re-apply the mutation against any later state change, so retries are unsafe and were disabled. The two systems coordinate over an unreliable channel without an end-to-end consensus mechanism, so a single dropped event = permanent state inconsistency.

Problem 3 - Synchronous LLM call is a single point of failure

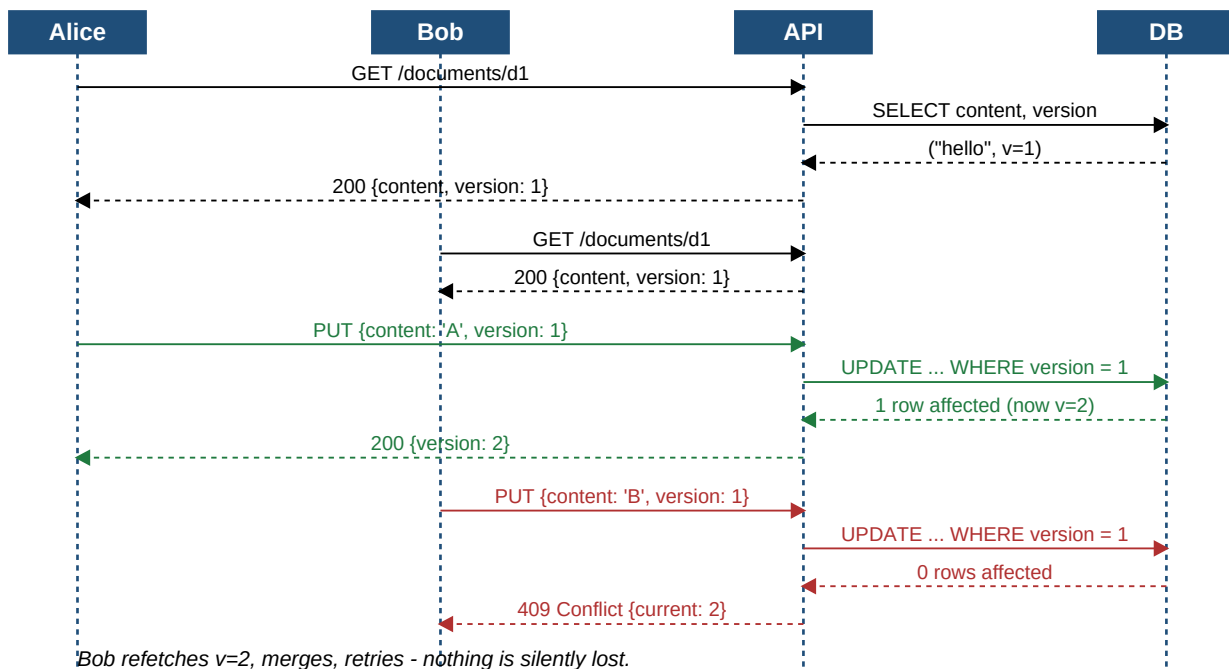
The LLM call is awaited inline inside the request handler with no timeout, no retry budget, and no bulkhead. When the upstream takes 60 seconds to time out at the TCP level, every in-flight request to that endpoint holds onto a worker / event-loop task for 60 seconds. Uvicorn's worker pool is finite, so once enough requests stack up, new requests for unrelated endpoints (like serving the React app) also queue. One unhealthy dependency cascades into total unavailability - the textbook definition of a SPOF made worse by tight coupling.

Part 2 - Design: making each failure mode survivable

Sync - Optimistic locking with monotonic version

Add a version INT column to every editable row. Clients GET the row with its version, send it back as part of the update payload, and the server runs UPDATE ... SET content=?, version=version+1 WHERE id=? AND version=?. Postgres returns 0 rows affected when the predicate fails; we translate that into **409 Conflict** with the current version embedded. The client refetches, performs a 3-way merge in the UI, and retries. For richer collaborative editing we could layer Operational Transformation or CRDTs over this, but for a study-doc app the OCC-with-merge-on-conflict approach is cheap, drift-free, and gives the user visible feedback rather than silent data loss.

Sequence diagram - two concurrent writers under OCC



Coordination - Idempotent webhook handler with retry queue and DLQ

Treat the webhook as an at-least-once delivery channel. (1) Verify Clerk's svix signature, (2) look up svix_id in a processed-events table; if present, return the original 200 immediately - this makes retries safe. (3) Apply the state change inside the same transaction that inserts the idempotency row, so either both happen or neither does. (4) If the handler raises, push the event onto a retry queue (Redis Streams or Postgres-based) with exponential backoff: 2s, 4s, 8s. (5) After MAX_RETRIES the event lands in a dead-letter queue for human triage instead of being silently dropped. A periodic reconciliation job can also pull current subscription state from the Clerk API and re-apply missed transitions - the belt-and-braces guarantee that we eventually converge even if a webhook is lost forever.

Fault tolerance - Circuit breaker + bounded timeout + fallback

Wrap the LLM call in a state machine: **CLOSED** forwards the call with a 1s timeout and counts failures; once *failure_threshold* consecutive failures trip it to **OPEN**, every subsequent call returns a cached/canned fallback in microseconds without touching the upstream. After *reset_after* seconds the breaker enters **HALF_OPEN** and lets one probe through; success closes the circuit, another failure re-opens it. This bounds the blast radius of an upstream outage from 'whole app hangs' to 'AI summaries return a graceful message'. Layered on top, a per-route `asyncio.Semaphore` (bulkhead) prevents the LLM endpoint from monopolising worker slots even before the breaker trips.

CAP trade-offs

Each fix picks a different point on the CAP/PACELC surface. **Sync (Problem 1)** chooses *Consistency over Availability*: we deliberately reject the stale writer with 409 instead of accepting both writes and merging silently - correctness of shared documents is non-negotiable. **Webhooks (Problem 2)** are inherently *eventually consistent*: we accept brief divergence between Clerk and our DB in exchange for at-least-once delivery and idempotent application - the system converges, just not instantaneously. **LLM (Problem 3)** chooses *Availability and bounded Latency over Consistency* with the upstream: we'd rather serve a slightly stale or canned response than block the user for 60 seconds. PACELC-wise: when the LLM partition heals, we still prefer latency over a strongly consistent freshness guarantee, so the breaker's half-open probe is the only call that pays the full upstream latency cost during recovery.